

QUDA MultiGrid 的算法理解与 PyQCU 复现

MATPC 紧凑 odd Schur、P/R、Clover/Gauge、33 点粗算子与 setup 优化

代码审计与代数验证

PyQCU · Lattice QCD

2026 年 8 月 30 日

本阶段结论：代数闭环与 MPI 闸门通过，setup 进入批量/显存优化

已完成

QUDA 风格 MATPC：细层一次 odd Schur；粗层直接 RDP。P/R、Clover/Gauge、33 点 stencil、V-cycle、重构与旧实现对照均已落地。

本轮证据

批量局部探测相对逐点为 $0.155\text{ s} \rightarrow 0.122\text{ s}$ ，约 $1.266\times$ ，最大差为 0；2/4 rank parity 回环最大差为 0。

资源策略

Schur 窗口矩阵按方向流式切片；工作区不足时当前批次自动减半重试。旧缺陷路径保留，不做无声替换。

完整 D_f

一次 S_o

P_0, R_0

$R_0 S_o P_0$

递归 V-cycle

阶段判断：关键不变量已经可测：奇偶布局保持、 $R = P^\dagger$ 、粗算子保留 local 与 hopping、分布式 33 点 halo 与混合精度运行不破坏真残差；当前主要瓶颈从正确性转为 setup 成本。

来源：实现：pyqcu/solver/_quda_multigrid.py:1094-1292, 1691-2331；优化：pyqcu/tools/_multigrid.py:938-1334；测试：examples/qcu/dev87/。

QUDA 的递归控制流: reset 建层, operator 做一次完整 V-cycle

输入: D_ℓ , B_ℓ , aggregate geometry, precision

reset: 生成/刷新 B_ℓ 与 Transfer_ℓ

$$B_{\ell+1,i} \leftarrow R_\ell B_{\ell,i}$$

createCoarseDirac: $D_{\ell+1} = R_\ell D_\ell P_\ell$

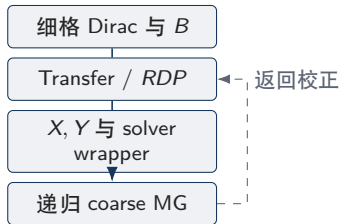
分别保存 residual、smoother 与 sloppy operator

createCoarseSolver: 封装 coarse solver 与递归 MG

operator: prepare $\rightarrow$ pre-smooth $\rightarrow Rr \rightarrow$ recurse

$\rightarrow Pe_c \rightarrow$ post-smooth

输出: solution; 若类型不同则 reconstruct 后重算 residual



控制流含义

粗化只改变表示空间; 每一级仍有完整 operator、平滑器、残差和停止判据。因而“粗层只有 dslash”不等价于 QUDA 的 coarse Dirac。



来源: QUDA: [refer/git-rep/quda/lib/multigrid.cpp](https://github.com/quda/lib/multigrid.cpp):72-197, 1131-1224; PyQCU: [pyqcu/solver/_quda_multigrid.py](https://github.com/pyqcu/solver/_quda_multigrid.py):1915-1986。

P/R 与粗算子：局部正交化与 RDP 缺一不可

Transfer 的物理/代数定义

$$\begin{aligned} a(x) &= (\lfloor x_0/b_0 \rfloor, \dots, \lfloor x_3/b_3 \rfloor), \\ (P\eta)_{sc}(x) &= \sum_j V_{scqj}(x) \eta_{qj}(a(x)), \\ (R\psi)_{qj}(a) &= \sum_{x \in a, s, c} V_{scqj}^*(x) \psi_{sc}(x), \\ R &= P^\dagger, \quad RP \simeq I_c. \end{aligned}$$

Wilson/Clover 的细自由度为 $D_f = 4 \times 3 = 12$;
每个 aggregate 内的 block CGS 产生 coarse
vectors V 。

对象	在 RDP 中的作用
Gauge U_μ	进入 Wilson forward/backward hopping; 方向、边界和伴随决定 $Y^\pm$ 。
Clover C	进入 local diagonal block X ; X^{-1} 参与 Schur 与预条件。
local block	$X(q) = V^\dagger D_{\text{local}} V$, 不是可省略的装饰项。
hopping block	$Y^\pm(q) = V^\dagger D_{\text{hop}} V$, 保留传播方向。
coarse stencil	$1 + 8 + 24 = 33$ 个本点、轴邻居和二维对角邻居。



奇偶处理: MATPC 首层形成 odd Schur, 粗层不再重复 checkerboard

块矩阵与重构

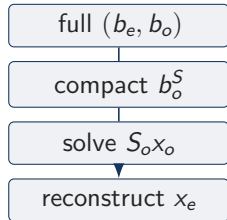
$$D = \begin{pmatrix} D_{ee} & D_{eo} \\ D_{oe} & D_{oo} \end{pmatrix}, \quad S_o = D_{oo} - D_{oe} D_{ee}^{-1} D_{eo}.$$

$$b_o^S = b_o - D_{oe} D_{ee}^{-1} b_e,$$

$$S_o x_o = b_o^S,$$

$$x_e = D_{ee}^{-1} (b_e - D_{eo} x_o).$$

D_{ee} 的 Clover/local block 必须可逆; $D^\dagger$ 路径需使用 $(D_{ee}^{-1})^\dagger$, 不能把 Schur 当作自动 Hermitian。



一次减半原则

$D_f \rightarrow S_o$ 已将 parity 压进 odd compact geometry; $A_{\ell+1} = R_\ell A_\ell P_\ell$ 直接 Galerkin, 不能再 Schur (否则 $T/4$)。

紧凑布局的坐标关系

$$p = (x + y + z + t) \bmod 2, \quad e_s = (x + y + z) \bmod 2,$$

$$t_{\text{full}} = 2t_{\text{half}} + ((p - x - y - z) \bmod 2).$$

预条件与 V-cycle: X^{-1} 改善局部尺度, 但不删除完整 D_c

粗层预条件的代数链

$$D_c = X + H,$$

$$D_{pc} = X^{-1}(D_c - X) = X^{-1}H,$$

$$D_{full,pc} = X^{-1}D_c = I + D_{pc},$$

$$\hat{Y}_\mu^+(q) = X^{-1}(q) Y_\mu^+(q).$$

backward link 要同时处理存储位置平移、伴随和右侧 $X^{-\dagger}$ 约定; 只检查 forward 不足以证明 PC 算子正确。

来源: QUDA: [refer/git-rep/quda/include/kernels/coarse_op_preconditioned.cuh](#), [refer/git-rep/quda/lib/dirac_coarse.cpp](#):351-430, 506-626; PyQCU: [pyqcu/solver/_quda_multigrid.py](#):912-1056, 2021-2034。

```

$$\begin{aligned} r_\ell &= b_\ell - A_\ell x_\ell \\ x_\ell &\leftarrow \text{smooth}(A_\ell, r_\ell) \\ r_{\ell+1} &\leftarrow R_\ell r_\ell \\ e_{\ell+1} &\leftarrow \text{solve}(A_{\ell+1}, r_{\ell+1}) \\ x_\ell &\leftarrow x_\ell + P_\ell e_{\ell+1} \\ x_\ell &\leftarrow \text{smooth}(A_\ell, r_\ell, x_\ell) \\ K(r_\ell) &= x_\ell \text{ 作为外层 FGMRES/BiCGStab 预条件器} \end{aligned}$$

```

实现分离

QUDA 同时维护 residual operator、smoother operator 与 sloppy precision; PyQCU 对应 `preconditioned_apply`、`v_cycle` 与显式 true residual。

PyQCU 的新旧实现边界：新路径复现 QUDA 代数，旧路径作为缺陷对照保留

维度	新实现: QudaMultigrid + CompactParityOperator	旧实现: solver.multigrid
finest operator	完整 Wilson/Clover D_f , 一次形成 S_o	MG-0 有奇偶处理, 但接口与新 compact 层级分离
coarse parity	当前 odd compact geometry 上直接 RDP, 不重复减半	粗层没有对应的完整 parity 递归约束
coarse operator	X local block + $Y^\pm$ hopping, 支持 33 点 stencil	粗层算子为不完备的 hopping/dslash 近似
P/R	aggregate-local block orthogonalization, $R = P^\dagger$	历史接口, 不能作为 QUDA 等价性证据
solve	V/W/F/K-cycle、smoother、reconstruct、true residual	作为历史/缺陷实现和对照基线

层级协议

$A_0 = D_f, A_1 = R_0 S_o P_0, A_{\ell+1} = R_\ell A_\ell P_\ell (\ell \geq 1)$ 。这就是“只在 finest 做 Schur”。

工程边界

新 API 通过 compact/full round-trip、Galerkin 和 QCU asset shape; 旧实现保留作缺陷对照, 不删除、不暗中替换。

来源: 新实现与旧实现的完整源码索引见附录; 对照矩阵为 `examples/qcu/dev87/comparison_matrix.md`。

局部 33 点 stencil setup: 用有限传播半径替代全格 Schur 探测

输入: fine Schur components, local orthogonal V , $W = 10$, E
按粗点 c 建窗口: $x_0 = 2c - (W/2 - 1)$ 在中心块填充 E 个单位探针
局部 action: $f_c \xrightarrow{D_{oe}} D_{ee}^{-1} \xrightarrow{D_{eo}} S_o f_c$
限制: 仅取 c 本点、4 轴邻居和 24 个二维对角块, 计算 $V^\dagger S_o V$
写回: sit , hop_nn , hop_diag ; 周期退化或窗口不足时回退逐点参考路径
输出: $[E, E, X_c, Y_c, Z_c, T_c] + [2, 4, E, E, \dots] + [2, 2, 6, E, E, \dots]$

为什么窗口足够

- $supp(D)$: 1 个轴向 hop,
- $supp(S_o)$: 两次 hop + local inverse,
- $supp(RS_o P)$: $c \pm 1$ 块与二维对角块.

$W = 10$ 给中心块外侧 padding, 周期边界用坐标模运算保持连续切片语义。



支撑边界

更宽 operator 必须扩展窗口与 C++ kernel, 禁止静默截断。

本轮性能/显存优化：计算批量化，同时把窗口矩阵从“全驻留”改为流式

峰值内存的量纲估算

旧路径在一次 Schur 调用前物化 18 组窗口矩阵：

$$M_{old} = 18K \cdot 12 \cdot 12 \cdot W^4 \cdot s_{cplx}.$$

$W = 10, K = 4$ 时, $M_{old} = 0.829 \text{ GB (c64)}, 1.659 \text{ GB (c128)}$, 尚未计入 Schur work buffers.

新路径

每个方向只保留 source、一个矩阵窗口和一次 contraction temporary; D_{ee}^{-1} 、 D_{oo} 分阶段切片, 并在 restrict 前释放局部场。

措施	工程效果
K 批量	K 个互不相交窗口一次调度, 减少 Python/CUDA launch 与切片开销。
流式矩阵	18 份矩阵不再同时存活, 峰值随当前方向而非 18 组增长。
及时释放	f_{local} 、 D_{local} 、restrict 临时在生命周期末立即释放。
OOM retry	当前批次按 $K \rightarrow \lceil K/2 \rceil$ 重试, 已写点不重复写。

可调旋钮

`build_stencil_local(..., batch_size=4)` 默认批量; 显存紧张可传更小 K , 异常时自动降档, 不改变 RS_oP 数学结果。

来源：实现：pyqcu/tools/_multigrid.py:938-987, 1026-1067, 1166-1240, 1279-1334; 测量命令：examples/qcu/dev87/test_stencil_batch_local.py 与本轮 CPU benchmark。

当前实测：批量局部探测保持逐点结果，并降低 setup 调度成本

验证项	参考/配置	当前结果	误差/状态	判读
流式 Schur vs 物化公式	随机非对角矩阵、 $K = 4$	shape 一致	$\max \text{ abs} = 0$	contraction 顺序与 parity mask 保持
局部 stencil $K=1$ vs $K=4$	fine 8^4 、coarse 4^4 、 $E = 3$	逐点/批量	$\max \text{ diff} = 0$	周期边界和 内部粗点均 覆盖
CPU batch benchmark	$E = 2, W = 6$ 、4 个粗 点	$0.155 \rightarrow 0.122 \text{ s}$	$1.266\times$	仅计 setup 探测，非 GPU 吞吐声 明
Python 主回归	pytest -q examples/pyqcu	26 passed	exit 0	既有代 数/solver 行 为未回归
breakdown guard	zero coarse operator	2 passed	finite output	粗求解 breakdown 不污染细解

数值不变量

代码审计与代数验证

逐点 vs 批量，E 相同，参考逐点结果相同：K 批量只改变独

口径

QUADA MG 复现与优化

CPU batch benchmark 用于证明调度收益和等价性：定数

分布式与混合精度验证：奇偶、粗 halo 和真残差均有独立证据

路径	配置	实测指标	结论
compact parity round-trip	2 rank [2, 1, 1, 1] c64; 2 rank [1, 1, 1, 2] c128	global max/rel = 0	$T/2$ 页按物理坐标重建正确
compact parity round-trip	4 rank [1, 2, 1, 2] c64; [1, 1, 2, 2] c128	global max/rel = 0	X/Y/Z/T rank 原点均覆盖
distributed coarse equivalence	2 rank、c64、[2, 1, 1, 1]	global L2 rel = 5.60×10^{-7}	33 点周期 halo 与全局参考一致
distributed coarse equivalence	2 rank、c128、[1, 1, 1, 2]	global L2 rel = 1.03×10^{-15}	双精度路径一致
coarse dslash smoke	2 rank、host staging、overlap off	rel max = 0	无死锁/非法访问
end-to-end MG	2 rank、fine c64/coarse c128	true residual rel $\simeq 7.69 \times 10^{-7}$	混合精度粗层可运行

MPI 设计判断

parity 页不能直接按压缩轴切块：先恢复物理格点、切 rank block、再压缩。粗层不做 checkerboard，但保留本 rank 的 33 点 stencil，并对跨 rank 的 32 个邻居做 halo。

来源：回环：examples/qcu/dev87/parity_mpi_roundtrip.py:48-118；粗算子：examples/qcu/dev87/coarse_mpi_equiv.py:46-122；MG：examples/qcu/dev87/run_qcu_mg_mpi.py:209-222；命令证据：本轮 MPI 运行输出。

当前限制与风险：正确性已闭环，生产性能仍受资产与通信口径约束

项目	已确证状态	仍需控制的风险/下一步
Schur window	$W = 10$ 对当前 Wilson/Clover 33 点支撑等价；边界有回退	更宽 dslash、非局部项需重新证明传播半径并扩大窗口
setup memory	矩阵切片流式化；K 批量工作区按需减半	fine operator 的大体量 Clover/Gauge 资产仍是主显存项，应继续 VRAM→RAM→HDF5 分层
MPI halo	2 rank 等价和 smoke 通过；当前为阻塞 host staging	未测通信/计算 overlap、device-aware MPI 和 NVSHMEM 吞吐
mixed precision	c64/c128 coarse transition 与 true residual 通过	误差平台、更多层级和更大体积需固定设备复测
QUDA 性能对照	算法控制流/代数对象完成映射	Python setup、QCU kernel、QUDA autotune/通信不是同一性能口径
legacy path	pyqcu/solver/_multigrid.py 保留作缺陷对照	使用新 API 时需显式选择，避免历史缓存与新 compact asset 混用

风险控制：所有“确证”只对应源码、恒等式或本轮命令输出；CPU batch 加速不外推 GPU；2/4 rank 正确性不外推强 scaling；未测项目保持“未验证”。

来源：边界实现：pyqcu/tools/_multigrid.py:1192-1222,1314-1327；历史资源分析：logs/dev80_2/dev80_2_analy.md:57-74；MPI 限制记录：logs/dev80_2/dev80_2_report.md:375-390。

阶段结论与下一步：从“能复现”转向“可扩展 setup”

已确认

QUDA 主线：

$B \rightarrow V \rightarrow P/R \rightarrow RDP$;

X 、 Y 、 X^{-1} 、 $Yhat$ 属于同一粗算子体系；

compact MATPC 只在 finest 做一次 odd Schur；

粗层完整保留 stencil 与 parity-aware 维度协议。

本轮交付

流式 BatchedLocalSchur、K 批量局部探测、OOM 降档重试；parity MPI round-trip 测试；流式/物化参考测试；本报告

..._20260830.tex/.pdf。

稳定性

build、Python、MPI coarse、MG breakdown 和混合精度闸门均为通过。

下一阶段验收

1. 记录真实 GPU peak memory 与 K 的曲线；
2. 大格 setup 的分层驻留和 HDF5 cache；
3. halo 通信与计算 overlap；
4. 同数据/同容差的 QUDA kernel benchmark。



来源：本轮证据：examples/qcu/dev87/；代码：pyqcu/tools/_multigrid.py:938-1334；数据读写目录：/root/PyQCU/data。

附录：对象—源码—证据索引

对象	QUDA 对照	PyQCU 当前证据
递归 MG geometry/spin map	lib/multigrid.cpp:72-197, 1131-1224 lib/transfer.cpp:200-257	QudaMultigrid.setup/v_cycle QudaTransfer, 含 Wilson/Clover map
P/R	lib/transfer.cpp:259-328	block CGS/QR、 $R = P^\dagger$ 、compact round-trip
coarse X/Y PC/Yhat parity	lib/coarse_op.cuh:1035-1459 lib/dirac_coarse.cpp:351-430, 506-626 lib/multigrid.cpp:1131-1224	lazy/materialized RDP、33 点资产 local inverse、preconditioned apply odd Schur、reconstruct、no repeated checkerboard
compact map	cpp/cuda/qcu/src/multigrid.cu: 1579-1743	CompactParityLayout、MPI round-trip
setup optimization	coarse_op.cuh 的局部支撑思想	BatchedLocalSchur 流式/K 批 量/OOM retry
旧实现	历史 PyQCU 路径	pyqcu/solver/_multigrid.py 保 留

证据等级

确证：本轮命令、数值直测或代码恒等式；**推断**：控制流与物理/代数对象的一致性判断；**未验证**：QUDA 原生 kernel 公平性能、device-aware overlap、NVSHMEM 与更大体积 setup 峰值。

附录：本轮验证命令与交付闸门

```
pytest -q examples/pyqcu
pytest -q examples/qcu/dev87/test_stencil_batch_local.py
examples/qcu/dev87/test_mg_breakdown.py
```

26 passed

2 passed

交付判定

页面数、渲染页数和逐页检查数必须一致；报告中的性能数字区分 CPU setup benchmark、CUDA/MPI 正确性和未验证的 QUDA 原生吞吐，不混用。

来源：文件与缓存：docs/report_quda_multigrid_reproduction_20260830.tex/.pdf；所有缓存读写遵循 /root/PyQCU/data。